

Лабораторна робота №10.

Розробка багатозадачної інтерактивної гри для міні-консолі

Мета роботи. Навчитися розробляти прості інтерактивні ігри типу “пінг-понг” та “змійка” для міні-консолі з використанням механізмів RTOS.

Хід виконання роботи.

1. Створіть проєкт FreeRTOS у середовищі розробки VS Code.
2. Завантажте в каталог вашого проєкту бібліотеки для роботи з LCD дисплеєм, перейшовши за посиланням:

<https://github.com/mpavlyk/rtos/blob/main/lib/lib2.zip>

3. Відкрийте файл **CMakeLists.txt**, який знаходиться у каталозі вашого проєкту і відредагуйте його наступним чином:

- знайдіть рядок із інструкцією `pico_sdk_init()`. Після цієї інструкції вставте:

1	# Add Subdirectory
2	add_subdirectory(/lib/Config)
3	add_subdirectory(/lib/OLED)
4	add_subdirectory(/lib/Fonts)
5	add_subdirectory(/lib/GUI)
6	add_subdirectory(/lib/LCD)
7	
8	# Include Directories
9	include_directories(/lib/Config)
10	include_directories(/lib/OLED)
11	include_directories(/lib/Fonts)
12	include_directories(/lib/GUI)
13	include_directories(/lib/LCD)

- Знайдіть інструкцію - `target_link_libraries(lab01 pico_stdlib)` і додайте бібліотеки - `hardware_spi`, `hardware_i2c`, `hardware_pwm`, `hardware_adc`, `Config`, `GUI`, `Fonts`, `OLED`.

1	target_link_libraries(lab
2	pico_stdlib
3	FreeRTOS-Kernel
4	FreeRTOS-Kernel-Heap4
5	hardware_spi

6	hardware_i2c
7	hardware_pwm
8	hardware_adc
9	hardware_pio
10	Config
11	GUI
12	Fonts
13	OLED
14	LCD)

4. Проаналізуйте, скомпілюйте та завантажте наведений код у Pico.

1	#include "DEV_Config.h"
2	#include "GUI_Paint.h"
3	#include "ImageData.h"
4	#include "Debug.h"
5	#include <stdlib.h>
6	
7	int OLED_1in3_C_test(void)
8	{
9	DEV_Delay_ms(100);
10	
11	printf("OELD_test Demo\r\n");
12	if(DEV_Module_Init()!=0){
13	while(1){
14	printf("END\r\n");
15	}
16	}
17	
18	/* Init */
19	OLED_1in3_C_Init();
20	OLED_1in3_C_Clear();
21	
22	UBYTE *BlackImage;
23	UWORD Imagesize = ((OLED_1in3_C_WIDTH%8==0)?
24	(OLED_1in3_C_WIDTH/8): (OLED_1in3_C_WIDTH/8+1)) *
25	OLED_1in3_C_HEIGHT;
26	if((BlackImage = (UBYTE *)malloc(Imagesize)) == NULL) {
27	while(1){
28	printf("Failed to apply for black memory...\r\n");
29	}
30	}
31	printf("Paint_NewImage\r\n");
32	Paint_NewImage(BlackImage, OLED_1in3_C_WIDTH, OLED_1in3_C_HEIGHT,
33	0, WHITE);

```

34
35
36
37 printf("Drawing\r\n");
38 //1.Select Image
39 Paint_SelectImage(BlackImage);
40 DEV_Delay_ms(500);
41 Paint_Clear(BLACK);
42
43 // 2.Drawing on the image
44 printf("Drawing:page 1\r\n");
45 Paint_DrawPoint(20, 10, WHITE, DOT_PIXEL_1X1, DOT_STYLE_DFT);
46 Paint_DrawPoint(30, 10, WHITE, DOT_PIXEL_2X2, DOT_STYLE_DFT);
47 Paint_DrawPoint(40, 10, WHITE, DOT_PIXEL_3X3, DOT_STYLE_DFT);
48 Paint_DrawLine(10, 10, 10, 20, WHITE, DOT_PIXEL_1X1,
49 LINE_STYLE_SOLID);
50 Paint_DrawLine(20, 20, 20, 30, WHITE, DOT_PIXEL_1X1,
51 LINE_STYLE_SOLID);
52 Paint_DrawLine(30, 30, 30, 40, WHITE, DOT_PIXEL_1X1,
53 LINE_STYLE_DOTTED);
54 Paint_DrawLine(40, 40, 40, 50, WHITE, DOT_PIXEL_1X1,
55 LINE_STYLE_DOTTED);
56 Paint_DrawCircle(60, 30, 15, WHITE, DOT_PIXEL_1X1,
57 DRAW_FILL_EMPTY);
58 Paint_DrawCircle(100, 40, 20, WHITE, DOT_PIXEL_1X1, DRAW_FILL_FULL);
59 Paint_DrawRectangle(50, 30, 60, 40, WHITE, DOT_PIXEL_1X1,
60 DRAW_FILL_EMPTY);
61 Paint_DrawRectangle(90, 30, 110, 50, BLACK, DOT_PIXEL_1X1,
62 DRAW_FILL_FULL);
63 BlackImage[0] = 0xf0;
64 // 3.Show image on page1
65 OLED_1in3_C_Display(BlackImage);
66 DEV_Delay_ms(2000);
67 Paint_Clear(BLACK);
68
69
70
71 // Drawing on the image
72 printf("Drawing:page 2\r\n");
73 Paint_DrawString_EN(10, 0, "Pico-OLED", &Font16, WHITE, BLACK);
74 Paint_DrawString_EN(10, 17, "hello world", &Font8, WHITE, BLACK);
75 Paint_DrawNum(10, 30, 123.1, &Font8, 2, WHITE, BLACK);
Paint_DrawNum(10, 43, 987654.2, &Font12, 2, WHITE, BLACK);
// Show image on page2
OLED_1in3_C_Display(BlackImage);
DEV_Delay_ms(2000);

```

```

Paint_Clear(BLACK);

// Drawing on the image
printf("Drawing:page 3\r\n");
Paint_DrawString_CN(10, 0,"Abc123", &Font12CN, WHITE, WHITE);
Paint_DrawString_CN(0, 20, "»¶ÓÊ'ÓÃ", &Font24CN, WHITE, WHITE);
// Show image on page3
OLED_1in3_C_Display(BlackImage);
DEV_Delay_ms(2000);

// Show image on page4
OLED_1in3_C_Display(gImage_1inch3_C_1);
DEV_Delay_ms(5000);

Paint_NewImage(BlackImage, OLED_1in3_C_WIDTH, OLED_1in3_C_HEIGHT,
180, WHITE);
Paint_Clear(BLACK);
int key0 = 15;
int key1 = 17;
int key = 0;
DEV_GPIO_Mode(key0, 0);
DEV_GPIO_Mode(key1, 0);

Paint_Clear(BLACK);
OLED_1in3_C_Display(BlackImage);

while(1){
    if(DEV_Digital_Read(key1 ) == 0){
        Paint_DrawRectangle(115, 5, 125, 15, WHITE,
DOT_PIXEL_2X2,DRAW_FILL_FULL);
        key = 1;
    }else {
        Paint_DrawRectangle(115, 5, 125, 15, BLACK,
DOT_PIXEL_2X2,DRAW_FILL_EMPTY);
    }

    if(DEV_Digital_Read(key0 ) == 0){
        Paint_DrawRectangle(115, 50, 125, 60, WHITE,
DOT_PIXEL_2X2,DRAW_FILL_FULL);
    }else {
        Paint_DrawRectangle(115, 50, 125, 60, BLACK,
DOT_PIXEL_2X2,DRAW_FILL_EMPTY);
        key = 1;
    }
}

```

```

    }
    if(key == 1){
        OLED_1in3_C_Display(BlackImage);
        Paint_Clear(BLACK);
    }

}

DEV_Module_Exit();
return 0;
}

```

5. Створіть задачу опитування кнопок та джойстика. Реалізуйте debounce (придушення брязкоту контактів). Передавайте значення стану кнопок та джойстика у чергу. Опитуйте кнопки та джойстик кожні 50 мс.
6. Створіть задачу ігрової логіки, яка буде реагувати на дії користувача та оновляти стан гри. Ігровий цикл повинен працювати з фіксованим часовим кроком, наприклад, 20–50 мс. Використовуйте `vTaskDelayUntil()` для фіксованого кадру.
7. Створіть задачу відображення кадрів на LCD дисплей. Використовуйте бінарний семафор для синхронізації кадру та mutex для буфера відображення.
8. Створіть головне меню в якому можна було вибрати гру та можливість завершити роботу.
9. Створіть меню, яке викликається при натисканні кнопки для паузи, де можна було вибрати, чи продовжувати гру, чи завершити гру.
10. Розробіть 2 – 3 гри для міні-консолі.
11. Перевірте роботу програми та оцініть реакцію на команди користувача в реальному часі.